3. Front-end com react

Sumário

3. Front-end com react 1
MÓDULO 3 – JavaScript Assíncrono e consumo de APIs 2
- Aula 1 – Introdução a APIs 2
- Aula 2 pt. 1 – Operações Assíncronas 2
- Aula 2 pt. 2 – Operações Assíncronas, Promises 6
- Aula 2 pt. 3 – Operações Assíncronas, async functions 8
- Aula 3 pt. 1 – Criando uma Requisição à API, Configurando o ambiente 11
- Aula 3 pt. 2 – Criando uma Requisição à API, GET 13
- Aula 3 pt. 3 – Criando uma Requisição à API, POST 20
- Aula 3 pt. 4 – Criando uma Requisição à API, PUT 27
- Aula 3 pt. 5 – Criando uma Requisição à API, DELETE 29

MÓDULO 3 – JavaScript Assíncrono e consumo de APIs

➢ Aula 1 – Introdução a APIs

API é um tipo de back-end que disponibiliza endpoints que recebem e retornam dados, normalmente em formato JSON. Isso permite que o front-end faça pedidos, ou seja, requisições, ao back-end por meio dessa interface.

Para exemplificar, vamos pensar que você queira se comunicar com um amigo, mas por algum motivo vocês nunca conseguem se encontrar pessoalmente. Então, vocês decidem que esse amigo vai disponibilizar uma espécie de caixa de correio compartilhada, que servirá tanto para o envio quanto para o recebimento das cartas. Agora, vamos definir algumas regrinhas para que essa comunicação funcione corretamente.

Primeiro: o seu amigo só pode responder às suas mensagens, ele nunca inicia uma conversa.
Segundo: a troca de mensagens será realizada apenas por meio de cartas.
Terceiro: todas as cartas que você enviar devem ser colocadas nessa mesma caixa de correio.
Quarto: as cartas com as respostas do seu amigo também serão colocadas nessa mesma caixa.
Quinto: você só pode enviar cartas dizendo como você está ou perguntando como ele está.

Definidas essas regras, vamos imaginar que você escreveu uma carta informando que está feliz por causa de um prêmio que recebeu e a colocou na caixa. O seu amigo pega a carta dessa mesma caixa, lê e responde algo como: “Fico feliz por você”, colocando a resposta novamente na caixa. Em outro momento, você pergunta como foi a semana dele, e ele responde que foi uma semana ótima, pois visitou os pais, novamente usando a mesma caixa.

Saindo desse exemplo e voltando para a explicação de API, você representa o front-end, que envia requisições e recebe as respostas. O seu amigo representa o back-end, que processa essas requisições e devolve as respostas. A caixa de correio representa a API, que é o único ponto de comunicação disponibilizado pelo seu amigo. É por meio dela que as requisições devem ser enviadas e as respostas recebidas, além de definir como essas informações devem ser trocadas. Em outras palavras, a API define onde e como o front-end pode se comunicar com o back-end.

➢ Aula 2 pt. 1 – Operações Assíncronas

No JavaScript, uma das coisas mais importantes são as operações assíncronas, mas antes de entrarmos nesse assunto, vamos entender o que seria uma operação síncrona.

Observe a Imagem 25. Nela, podemos observar que temos três `console.log()` e o resultado da execução desse programa aparece na Imagem 26. Podemos ver que os números que aparecem são 1, 2 e 3, nessa ordem. Esse é o comportamento esperado na

execução de um código síncrono: o programa é executado na ordem em que os códigos foram escritos.

```
JS exemple-real-00.js ×
JS exemple-real-00.js
1   console.log(1)
2
3   console.log(2)
4
5   console.log(3)
6
7
```

Imagem 25: Código síncrono

```
PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_3\event-loop> node .\exemple-real-00.js
1
2
3
```

Imagem 26: Resultado do código síncrono

Agora observe a Imagem 27. Note que colocamos o segundo `console.log()` dentro da função `setTimeout()`. Essa função recebe como primeiro parâmetro uma função e, como segundo parâmetro, a quantidade mínima de milissegundos que a função do primeiro parâmetro deve esperar para ser executada. Neste caso, a função do primeiro parâmetro deve ser executada após, no mínimo, dois segundos.

```
JS exemple-real-01.js ×
JS exemple-real-01.js > ...
1   console.log(1)
2
3   setTimeout(() => console.log(2), 2000)
4
5   console.log(3)
6
7
```

Imagem 27: Código assíncrono

O comportamento esperado para esse programa seria aparecer no console o número 1, depois de 2 segundos aparecer o número 2 e, por último, aparecer o número 3. Observe a Imagem 28. com o resultado no console.

```
PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_3\event-loop> node .\exemple-real-01.js
1
3
2
```

Imagem 28: Resultado no console

A ordem que aparece é 1, 3 e 2. Vale destacar que o 2 aparece depois de um tempo por causa do segundo parâmetro da função `setTimeout()`. Mas você pode estar se perguntando o porquê disso ter acontecido.

Isso aconteceu por causa do assincronismo no JavaScript, mas por que isso existe?

O JavaScript é uma linguagem de programação single-thread e, por isso, só consegue executar uma instrução por vez. No entanto, pode acontecer de uma dessas instruções depender de algo que esteja fora do seu programa. Lembre que, na aula anterior, falamos sobre o que é uma API e como ela é uma interface de comunicação entre o front-end e o back-end. Então pense comigo: você tem total controle dos dados exclusivos do front-end, mas não tem nenhum controle sobre o tempo que vai levar até o back-end responder à sua requisição.

Isso gera um grande problema, pois, se o JavaScript só consegue executar uma instrução por vez e ele tiver que esperar por um tempo indeterminado até o back-end responder à sua requisição, o que pode acontecer em milissegundos ou em horas, o seu programa vai ficar parado, sem processar outros comandos que não dependem dos dados do back-end.

É por isso que surgiu a assincronicidade, mas como ela funciona?

Observe a Imagem 29, ela explica o processo de execução de um programa em JavaScript. Nessa explicação, vamos dar mais foco à Call Stack, ao Event Loop e à Callback Queue. Os outros componentes que aparecem na Imagem são o Heap, que é um lugar na memória onde ficam armazenados os objetos do seu programa, e as Web APIs, que são APIs utilizadas no front-end. Note que o Timeout que aparece nas Web APIs representa o setTimeout que foi utilizado no exemplo anterior, pois ele não faz parte do núcleo da linguagem JavaScript, ECMAScript, mas sim uma funcionalidade fornecida pelo ambiente de execução, como o navegador ou, no nosso caso, o Node.js. Por ser algo com que o ambiente deve se preocupar, ele é considerado uma Web API.

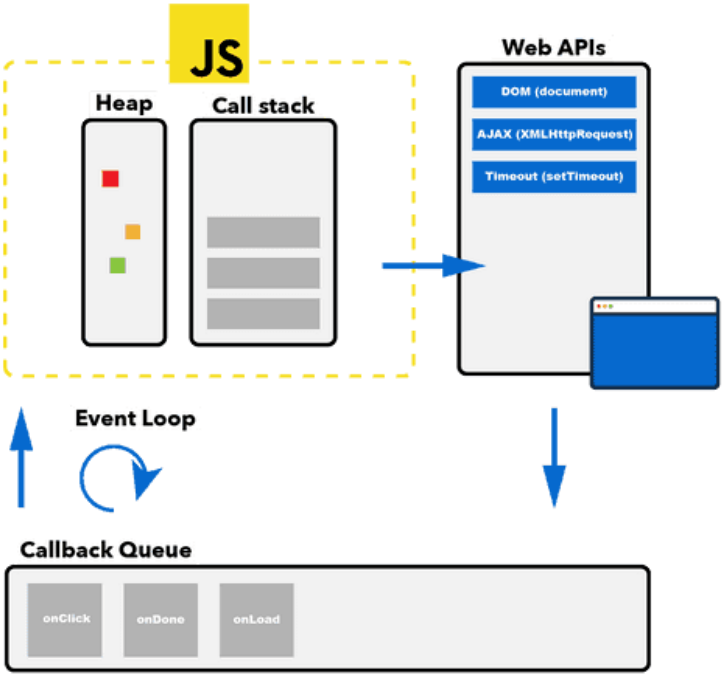

Imagem 29: Processo de execução em JavaScript

Vamos começar pela Call Stack, ou pilha de chamadas. Pense que todas as funções que você escreve, em algum momento, vão ser executadas na Call Stack. A Call Stack funciona como uma pilha, seguindo a lógica LIFO, Last In, First Out, ou seja, o último elemento a entrar é o primeiro a sair.

Se uma função que já está na Call Stack chamar outra função dentro dela, essa nova função será empilhada em cima da primeira. A execução passa então para essa segunda função. Quando ela termina sua execução, ela é removida da Call Stack e o controle volta para a função anterior, exatamente na linha onde a segunda função foi chamada. A execução continua a partir desse ponto até que não haja mais nenhuma instrução a ser executada nessa função. Somente depois que ela terminar completamente é que será removida da Call Stack.

No nosso primeiro exemplo desta aula, foram os três `console.log()`. Então, a primeira função a ser executada na Call Stack seria o `console.log(1)` e, como essa é uma função síncrona, ela é executada imediatamente, fazendo aparecer no console o valor 1. Em seguida, essa função, que já terminou a execução, é retirada da Call Stack. A próxima função a entrar na Call Stack é o `console.log(2)` e, como também é síncrona, ela é executada e aparece o valor 2 na tela. Depois, a função que já terminou a execução sai da Call Stack. O mesmo processo acontece com a função `console.log(3)`.

Já no segundo exemplo, a primeira função que vai para a Call Stack é o `console.log(1)`, e isso segue a mesma execução do exemplo anterior. Depois, o `setTimeout()` vai para a Call Stack e, como já discutimos, é uma função assíncrona. Por isso, a sua execução acontece de forma diferente. Por ser uma função de Web API, ela vai ser executada no ambiente Node.js. Se fosse uma requisição a uma API de um back-end, essa requisição

seria executada nesse back-end. Depois disso, a Call Stack recebe a função `console.log(3)` e, por ela ser síncrona, é executada, seu valor é impresso no console e a função sai da Call Stack.

Ao mesmo tempo em que a execução de `console.log(3)` acontecia na Call Stack, o ambiente de execução estava executando o `setTimeout()`, que consiste em esperar dois segundos. Passados esses dois segundos, o ambiente envia a função callback `console.log(2)` para a Callback Queue, ou fila de callbacks. É uma fila porque os primeiros elementos a entrar são os primeiros a sair. Observação: função callback, em JavaScript, é uma função que é passada como parâmetro para outra função e deve ser chamada em algum momento dentro ou depois que essa segunda função for executada.

Agora entra uma das partes mais importantes: o Event Loop, ele é um loop que fica verificando se a Call Stack está vazia. Caso esteja, significa que as funções síncronas já foram executadas e, com isso, ele vai até a Callback Queue, retira a primeira função que entrou e a coloca na Call Stack para ser executada. Nesse caso, é a função `console.log(2)`, que finalmente é executada, tem seu resultado impresso no console e, por fim, sai da Call Stack, deixando-a vazia novamente.

Uma observação importante é que a Callback Queue é dividida em duas filas diferentes: a fila que armazena tasks e a fila que armazena microtasks. Em tasks ficam armazenadas funções como setTimeout, setInterval, setImmediate, operações de entrada e saída, entre outras. Já nas microtasks ficam as Promises, async functions, process.nextTick() e queueMicrotask. As microtasks têm mais prioridade que as tasks, ou seja, são executadas antes das tasks.

Com esse mecanismo, o programa em JavaScript não fica esperando a resposta de uma API por tempo indeterminado enquanto ainda existem outros comandos que podem ser executados.

- **Aula 2 pt. 2 – Operações Assíncronas, Promises**

Promises são objetos do JavaScript que representam um valor que pode estar disponível agora, no futuro ou nunca, caso ocorra um erro. Elas são muito usadas para lidar com operações assíncronas, como requisições HTTP, leitura de arquivos ou timers. No front-end, é mais comum consumirmos Promises do que criá-las manualmente, mas entender como elas funcionam é fundamental.

Na Imagem 30, temos:

```
JS promise.js ×
JS promise.js > ...
1  p = new Promise((resolve) => {
2      console.log("Dentro da promise")
3      resolve("tudo ok")
4  })
5  p.then((res) => console.log("tratando o valor: ", res))
6  p.catch((error) => console.log(error))
7
8  console.log("Inicio")
9
```

Imagem 30: Exemplo de Promises

No código apresentado, a variável p recebe uma nova Promise. O construtor `new Promise()` recebe como argumento uma função chamada executor. Essa função é executada imediatamente no momento em que a Promise é criada, ou seja, sua execução é síncrona. O executor recebe como parâmetro a função `resolve`, que é responsável por marcar a Promise como concluída com sucesso.

A função `resolve` pode receber qualquer tipo de valor, como uma string, número, objeto ou array. Esse valor será disponibilizado posteriormente para tratamento nos métodos `then`. O método `then` recebe uma função que será executada quando a Promise for resolvida e é importante destacar que podemos ter mais de um `then`. Quando encadeamos vários `then`, cada um pode transformar o valor recebido e retornar um novo valor, que será passado para o próximo `then` da cadeia. Isso permite criar fluxos assíncronos organizados e sequenciais.

Já o método `catch` é utilizado para tratar erros, seja um erro lançado dentro do executor, seja um erro ocorrido em algum `then`. Ele também recebe uma função como parâmetro, que será chamada caso a Promise seja rejeitada. Além disso, quando usamos encadeamento de `then`, um único `catch` ao final da cadeia é capaz de capturar qualquer erro que tenha ocorrido em qualquer etapa anterior.

Sobre a ordem de execução, ao rodar esse código o que aparece no console é, conforme mostrado na Imagem 31, primeiro a mensagem "Dentro da promise", depois "Inicio" e, por último, "tratando o valor: tudo ok". A imagem ilustra exatamente essa sequência de saída no terminal.

```
PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_3\async-functions> node .\promise.js
Dentro da promise
Inicio
tratando o valor:  tudo ok
```

Imagem 31: Execução do exemplo de Promises

Isso acontece porque o código dentro do executor da Promise é executado imediatamente, de forma síncrona, no momento em que a Promise é criada. Ou seja, assim que o JavaScript encontra o `new Promise(...)`, ele já executa o `console.log("Dentro da promise")`.

Em seguida, o `console.log("Inicio")` também é executado de forma síncrona, pois faz parte do fluxo principal do programa e ainda está na call stack.

Somente depois que todo o código síncrono termina é que a função passada para o `then` é executada. Mesmo que o `resolve("tudo ok")` tenha sido chamado imediatamente, o callback do `then` é colocado na fila de microtasks e só é processado quando a pilha de execução estiver vazia

Portanto, embora o executor da Promise seja síncrono, os tratamentos registrados com `then` e `catch` são sempre assíncronos. Essa diferença é fundamental para entender por que a ordem de execução ocorre dessa maneira e para evitar erros ao trabalhar com código assíncrono em JavaScript.

➢ Aula 2 pt. 3 – Operações Assíncronas, async functions

As async functions são utilizadas para pausar a execução de uma função para esperar o valor de uma Promise e, depois de receber esse valor, continuar a execução do restante da função. Vale ressaltar que async functions sempre retornam uma Promise, mesmo que ela não tenha sido escrita explicitamente na função.

A Imagem 32 ilustra uma async function.

Imagem 32: Exemplo de async functions

A maior diferença entre o código da Imagem 30 e o da Imagem 32 é que, na Imagem 32, envolvemos a Promise dentro de uma async function e atribuímos o valor retornado a uma constante (const) chamada p. Esse código executa exatamente da mesma maneira que o anterior. A diferença é que, nesse caso, conseguimos escrever then e catch de outra forma.

Nas async functions podemos usar a palavra reservada await antes de uma Promise. Com isso, podemos remover o then e simplesmente escrever, depois da Promise, o código que antes ficaria dentro do then. Por enquanto, vou comentar o catch. Observe a Imagem 33.

```
JS exemple-01.js ×
JS exemple-01.js > exemplo
1   async function exemplo() {
2       const p = await new Promise((resolve) => {
3           console.log("Dentro da promise")
4           resolve("tudo ok")
5       })
6       console.log("tratando o valor: ", p)
7       // p.catch((error) => console.log(error))
8   }
9
10  exemplo()
11  console.log("Inicio")
```

Imagem 32: Utilizando a palavra reservada await

Observe que, ao retirar o then, o parâmetro que foi passado para a função resolve não é mais enviado para o then, e sim para a constante p. Com a exceção do catch comentado, esse código é exatamente igual ao anterior.

Sem a presença do await antes da Promise, caso a sua resolução dependa de algum código assíncrono, o valor de p ainda não estará disponível no momento em que console.log("tratando o valor", p) for executado.

Isso acontece porque o JavaScript não entende que esse console.log deve ser executado após a resolução da Promise, como aconteceria dentro de um then. Dessa forma, o console.log é executado imediatamente, antes da Promise terminar, e o valor de p será apenas uma Promise pendente em vez do resultado esperado.

Observe as Imagens 33 e 34 com o código sem o await e o resultado no terminal.

```
JS exemple-01.js ×
JS exemple-01.js > exemplo > p
1   async function exemplo() {
2       const p = new Promise((resolve) => {
3           console.log("Dentro da promise")
4           setTimeout(() => resolve("tudo ok"), 2000)
5       })
6       console.log("tratando o valor: ", p)
7       // p.catch((error) => console.log(error))
8   }
9
10  exemplo()
11  console.log("Inicio")
```

Imagem 33: Código sem await

```
PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_3\async-functions> node .\exemple-01.js
Dentro da promise
tratando o valor:  Promise { <pending> }
Inicio
```

Imagem 34: Resultado no console

Podemos observar que no console aparece tratando o valor: Promise { <pending> } antes de Início, e o valor de p aparece como uma Promise pendente. Isso acontece porque console.log("tratando o valor:", p) foi executado antes do término da Promise, já que o JavaScript não interpretou esse trecho de código como um then.

Agora observe as Imagens 35 e 36 com o código com o await e o resultado no terminal.

```
JS exemple-01.js ×
JS exemple-01.js > exemplo > p
1   async function exemplo() {
2       const p = await new Promise((resolve) => {
3           console.log("Dentro da promise")
4           setTimeout(() => resolve("tudo ok"), 2000)
5       })
6       console.log("tratando o valor: ", p)
7       // p.catch((error) => console.log(error))
8   }
9
10  exemplo()
11  console.log("Inicio")
```

Imagem 35: Código com await

```
PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_3\async-functions> node .\exemple-01.js
Dentro da promise
Inicio
tratando o valor:  tudo ok
```

Imagem 36: Resultado no terminal

Agora, mesmo que a Promise demore para ser resolvida, o código console.log("tratando o valor:", p) só será executado quando ela terminar.

Voltando agora para o catch, onde podemos colocar o código para tratar os erros da nossa async function?

Muito simples. Basta envolver o código que você espera que funcione corretamente dentro de um try e colocar, dentro do catch, o código que deve ser executado caso ocorra algum erro dentro do bloco try. Observe a Imagem 37.

```
JS exemple-01.js ×
JS exemple-01.js > ...
1   async function exemplo() {
2       try {
3           const p = await new Promise((resolve) => {
4               console.log("Dentro da promise")
5               setTimeout(() => resolve("tudo ok"), 2000)
6           })
7           console.log("tratando o valor: ", p)
8       }catch (error) {
9           console.log(error)
10      }
11  }
12
13  exemplo()
14  console.log("Inicio")
```

Imagem 37: Uso de try/catch

Com isso, encerramos as aulas voltadas ao entendimento do assincronismo em JavaScript. Ao longo dessas aulas, foram apresentados os principais conceitos teóricos que permitem compreender como o JavaScript lida com operações assíncronas, como Promises, then, catch, async e await.

Nas próximas aulas, iremos aplicar esses conceitos na prática, aprendendo a fazer requisições para APIs. Utilizaremos como base todo o conteúdo teórico abordado até aqui para entender melhor como o JavaScript realiza chamadas assíncronas para servidores, recebe dados e os utiliza dentro das aplicações. Dessa forma, será possível conectar a teoria estudada com situações reais de desenvolvimento.

➢ Aula 3 pt. 1 – Criando uma Requisição à API, Configurando o ambiente

Depois de estudarmos o que é uma API e como funcionam as funções assíncronas em JavaScript, nesta aula vamos configurar um exemplo de CRUD, ou seja, criar, ler, atualizar e deletar dados. Nas próximas aulas vamos estudar como obter dados de uma API utilizando o método GET, inserir dados com o método POST, realizar alterações com o método PATCH e apagar registros com o método DELETE.

Para iniciar o estudo, precisamos criar um novo projeto React. Podemos chamar esse projeto de “requisicao-api”. Recomendo que você retorne às aulas do módulo 2 para acompanhar melhor esta etapa. Como o foco aqui não será o CSS, vamos excluir o arquivo index.css e remover a importação desse arquivo no main.jsx. Além disso, será necessário apagar o conteúdo do arquivo App.jsx que está dentro de `<>...</>`. Observe a Imagem 38, que mostra como o arquivo App.jsx deve ficar.

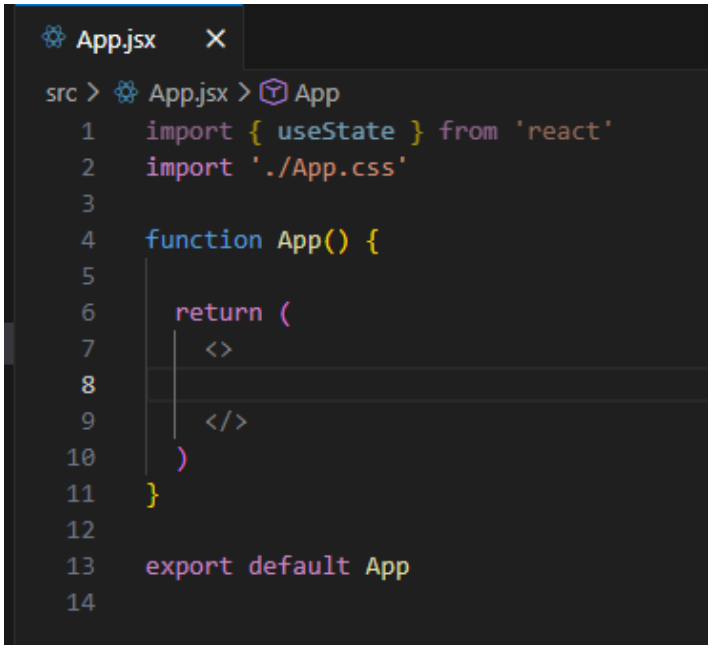

Imagem 38: Arquivo App.jsx

Na parte superior esquerda da tela do VS Code, podemos encontrar a opção **Terminal** no menu. Clique nela e, em seguida, na opção **Novo Terminal**, como mostrado na Imagem 39.

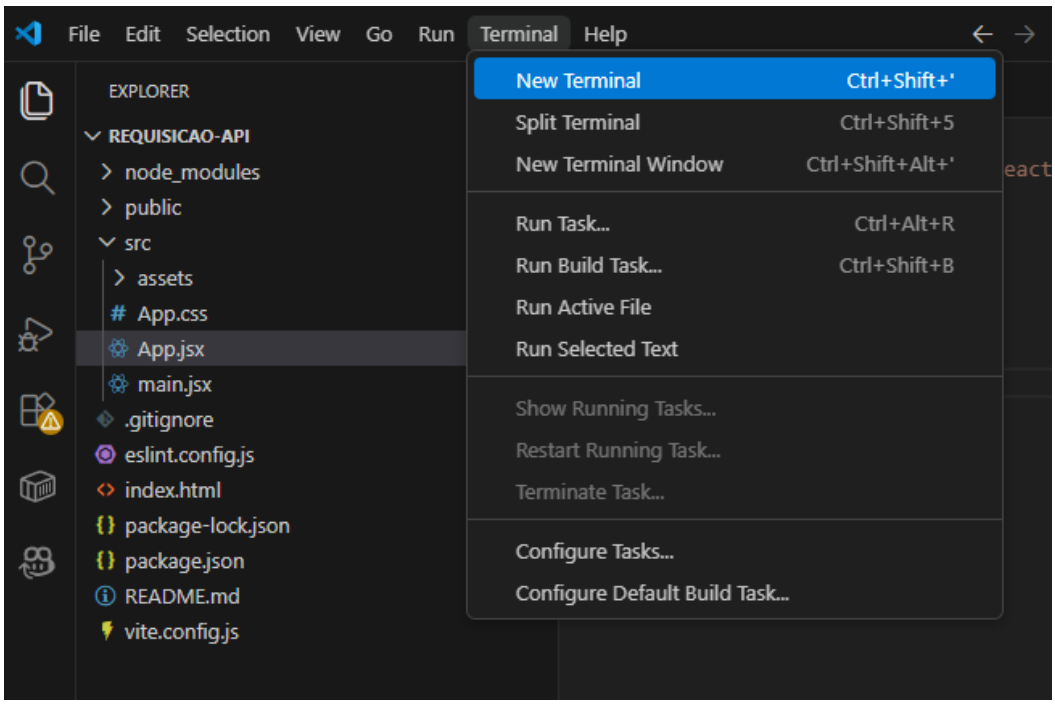

Imagem 39: Abrindo um novo terminal

Quando o terminal aparecer na parte inferior do VS Code, digite o comando `npm run dev` para iniciar o projeto.

Agora vamos importar algumas bibliotecas que serão importantes para o desenvolvimento deste projeto. Abra um novo terminal novamente e execute o comando abaixo:

```
npm install axios
```

Em seguida, execute também o comando:

```
npm install json-server
```

Crie um arquivo chamado `db.json` na pasta raiz do projeto. Por fim, execute o comando abaixo no terminal para iniciar o json-server, utilizando o arquivo `db.json` como base de dados para as requisições.

```
npx json-server db.json
```

A biblioteca axios será responsável por permitir que façamos as requisições. Já a biblioteca json-server será utilizada para simular um back-end e possibilitar modificações no arquivo `db.json`, que representará o nosso banco de dados.

➢ Aula 3 pt. 2 – Criando uma Requisição à API, GET

Agora que já temos a configuração inicial do nosso exemplo, podemos começar a fazer as requisições.

Como o método POST, responsável por inserir dados, ainda não foi implementado, precisaremos adicionar alguns dados manualmente no arquivo `db.json`. Lembre-se de que, caso esteja realizando esta aula depois de ter fechado o projeto criado na aula anterior, será necessário iniciar novamente o front-end com `npm run dev` e a nossa simulação de servidor e banco de dados com `npx json-server db.json`.

Vamos criar uma lista de usuários no arquivo `db.json`. Cada usuário terá um id, um nome e sua idade. Observe a Imagem 40, podemos ver que cada pessoa na lista de usuários é representada por um objeto com id, nome e idade. O id deve ser único para cada pessoa, pois é ele que utilizaremos para identificar um usuário específico, funcionando de maneira semelhante a um CPF.

Observe também que as chaves id, nome e idade devem estar entre aspas duplas, assim como os valores do tipo string, por exemplo, "marcos" e "samuel". Já os valores numéricos não precisam estar entre aspas.

Imagem 40: Registros no arquivo db.json

Vou utilizar, neste exemplo, um hook que será explicado com mais detalhes no próximo módulo, chamado `useState`. Porém, não precisa se preocupar agora. Pense apenas que ele possui um valor inicial, uma variável responsável por armazenar e mostrar esse valor e uma função que permite alterar o valor dessa variável. Observe abaixo como ele é criado:

```
const [valor_atual, funcao_pra_mudar_valor_atual] =
useState(valor_inicial)
```

Para que esse hook possa ser utilizado, é necessário importá-lo do React com o comando no início do arquivo, no nosso caso, `App.jsx`:

```
import { useState } from 'react'
```

No nosso exemplo, o hook representará uma lista de usuários. Seu valor inicial será uma lista vazia. Por convenção, o nome da função responsável por alterar o valor do hook começa com `set`, seguido do nome do hook com a primeira letra maiúscula.

Veja o código abaixo e, em seguida, observe a Imagem … nas linhas 1 e 7:

```
const [usuarios, setUsuarios] = useState([ ])
```

```
App.jsx   db.json
src > App.jsx > App
1  import { useState, useEffect } from 'react'
2  import axios from 'axios'
3  import './App.css'
4
5  function App() {
6
7    const [usuarios, setUsuarios] = useState([])
8
9
10
11   return (
12     <>
13
14     </>
15   )
16 }
17
18 export default App
19
```

Imagem 41: Importação e utilização do hook useState

Vamos iniciar nossa requisição GET dentro de outro hook chamado `useEffect`. Não se preocupe muito com ele agora, pois haverá uma aula específica no próximo módulo explicando seu funcionamento com mais detalhes. Nesta aula não entraremos em muitos detalhes sobre esse hook.

Para utilizá-lo, precisamos importá-lo do React. Portanto, vá até a importação do `useState`, adicione uma vírgula e importe também o `useEffect`. Esse hook recebe como primeiro parâmetro uma função que será executada e, como segundo parâmetro, uma lista de hooks ou props. Sempre que algum elemento dessa lista for alterado, a função passada no primeiro parâmetro será executada novamente. Como queremos que essa função seja executada apenas na primeira renderização, deixamos a lista vazia.

Observe as linhas de 9 a 11 da Imagem …

É dentro da função passada como primeiro parâmetro do `useEffect` que escreveremos nossa função assíncrona para realizar a requisição GET, que irá buscar os usuários armazenados no arquivo `db.json`. Essa função precisa ser assíncrona porque utilizaremos `await` para aguardar a resposta do axios, garantindo que as linhas de código abaixo só sejam executadas depois que a resposta for recebida.

Também precisamos importar o axios no início do arquivo com o código abaixo. Observe a Imagem 42 na linha 2:

```
import axios from 'axios'
```

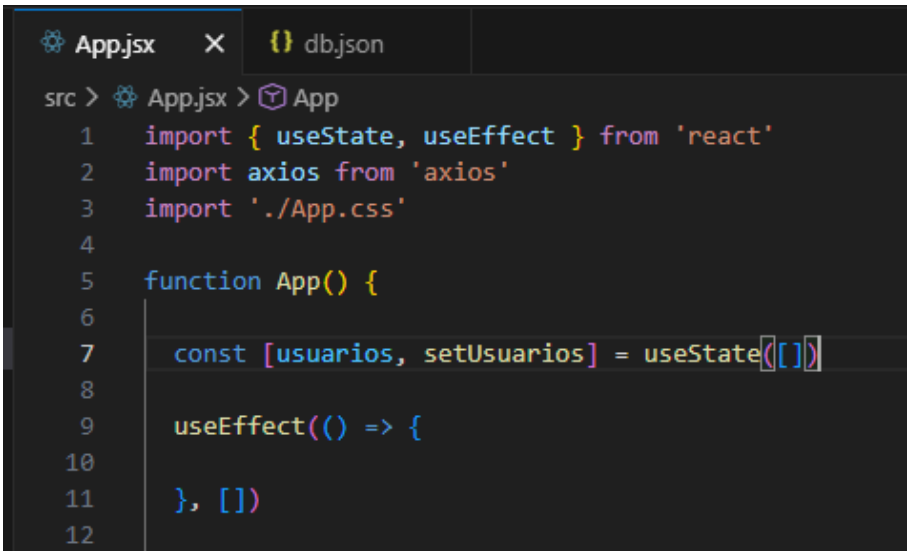

Imagem 42: Importação da biblioteca axios

Observe a Imagem 43 e a explicação abaixo.

Para começar, envolvemos o código da requisição dentro da nossa função assíncrona utilizando try e catch. No bloco catch colocamos um `console.error` para que, caso ocorra algum erro, ele seja exibido no console.

Fazemos a requisição utilizando o axios e utilizamos sua função GET para realizar a requisição. A função GET deve receber como primeiro parâmetro a URL que irá retornar os nossos dados. A URL que o json-server utiliza é `http://localhost:3000`. Nela, http é o protocolo, localhost indica que o json-server está sendo executado no nosso próprio computador e 3000 é a porta na qual ele está escutando as requisições. Como queremos obter os dados dos usuários, precisamos acrescentar isso à URL. Assim, sua versão final será:

```
http://localhost:3000/usuarios
```

O resultado da requisição será armazenado na constante resposta. Desses dados, utilizaremos a propriedade data, que contém efetivamente as informações dos usuários. Esses dados devem ser armazenados na lista criada com `useState` chamada `usuarios`. Para isso, utilizamos a função `setUsuarios` e passamos como parâmetro resposta.data.

```
App.jsx   db.json
src > App.jsx > App
1   import { useState, useEffect } from 'react'
2   import axios from 'axios'
3   import './App.css'
4
5   function App() {
6
7     const [usuarios, setUsuarios] = useState([])
8
9
10    useEffect(() => {
11      async function buscarUsuarios() {
12        try {
13          const resposta = await axios.get("http://localhost:3000/usuarios")
14          setUsuarios(resposta.data)
15        } catch (error) {
16          console.error(error)
17        }
18      }
19
20      buscarUsuarios()
21    }, [])
```

Imagem 43: Uso do método GET

Com isso, o front-end já tem acesso aos dados dos usuários. Agora precisamos exibir esses dados na tela. Para isso, vamos modificar o JSX no retorno do arquivo `App.jsx`.

Observe a Imagem 44, nela é utilizado o método `map` na lista de `usuarios` dentro do retorno do componente App. A função map, no JavaScript, é responsável por percorrer uma lista. Como parâmetro dessa função, é passada outra função que define o que será feito com cada elemento da lista.

Na função que lida com cada elemento da lista, foi passado como parâmetro o usuário. Em seguida, foi criada uma `div` contendo três parágrafos para exibir o id, o nome e a idade do usuário recebido como parâmetro.

O parâmetro `key`, utilizando o id do usuário, serve para que o `map` consiga identificar cada `div` de forma única.

```jsx
import { useState, useEffect } from 'react'
import axios from 'axios'
import './App.css'

function App() {

  const [usuarios, setUsuarios] = useState([])

  useEffect(() => {
    async function buscarUsuarios() {
      try {
        const resposta = await axios.get("http://localhost:3000/usuarios")
        setUsuarios(resposta.data)
      } catch (error) {
        console.error(error)
      }
    }

    buscarUsuarios()
  }, [])

  return (
    <>
        {usuarios && usuarios.map((usuario) => (
          <div key={usuario.id}>
            <p>Id: {usuario.id}</p>
            <p>Nome: {usuario.nome}</p>
            <p>Idade: {usuario.idade}</p>
          </div>
        ))}
    </>
  )
}

export default App
```

Imagem 44: Exibição de dados dos usuários

Se olharmos no navegador, podemos ver que os usuários já estão sendo exibidos na tela. Observe a Imagem 45.

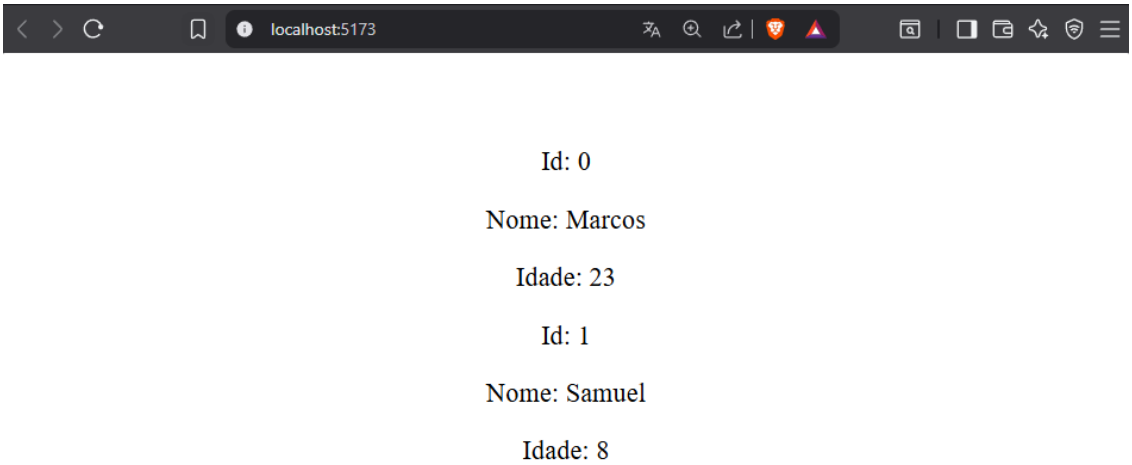

Imagem 45: Resultado no navegador

Apenas para fins estéticos, podemos fazer algumas alterações no código.

Observe a Imagem 46. Nela, envolvemos o nosso `map` em uma `div` e adicionamos um `classname` chamado `usuarios`. É por meio dessa classe que iremos fazer alterações no arquivo `App.css`. Já na `div` criada dentro do `map`, foi adicionado um `classname` chamado `usuario` para realizar as configurações de estilo de cada usuário.

```
22    return (
23      <>
24        <div className="usuarios">
25          {usuarios && usuarios.map((usuario) => (
26            <div className='usuario' key={usuario.id}>
27              <p>Id: {usuario.id}</p>
28              <p>Nome: {usuario.nome}</p>
29              <p>Idade: {usuario.idade}</p>
30            </div>
31          ))}
32        </div>
33      </>
34    )
35  }
```

Imagem 46: Classes para melhorar a legibilidade dos dados

No arquivo `App.css`, foram feitas as alterações mostradas na Imagem 47.

```
App.jsx    # App.css    {} db.jso
src > # App.css > .usuarios
1   #root {
2     max-width: 1280px;
3     margin: 0 auto;
4     padding: 2rem;
5     text-align: center;
6   }
7
8   .usuarios {
9     display: flex;
10    justify-content: center;
11    gap: 20px;
12    flex-wrap: wrap;
13  }
14
15  .usuario {
16    border: 1px solid black;
17    padding: 0px 5px;
18  }
```

Imagem 47: Arquivo App.css

Quando abrimos novamente o navegador, temos como resultado o que está mostrado na Imagem 48, com cada usuário exibido lado a lado em um pequeno quadrado.

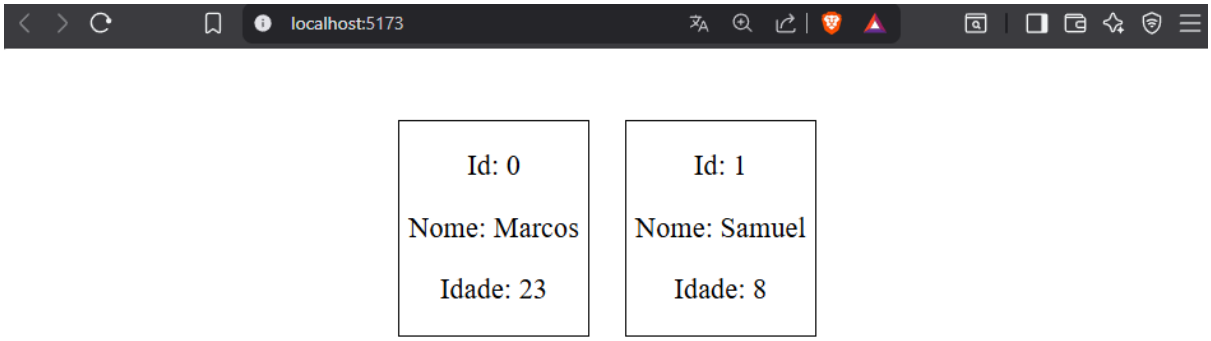

Imagem 48: Resultado no navegador

Fique à vontade para modificar o código e o estilo dos usuários para explorar novas possibilidades.

- **Aula 3 pt. 3 – Criando uma Requisição à API, POST**

Agora que já aprendemos como obter os dados armazenados utilizando o método GET, vamos aprender como inserir dados usando o método POST.

Para inserir dados, precisamos de um formulário para que o usuário da aplicação possa digitar as informações. Observe a Imagem 49. Nela foi inserido o código de um formulário no início do retorno do JSX. Esse formulário possui um input do tipo text com o `name` nome para o nome do usuário, um input do tipo number com o `name` idade para a idade do usuário e um `input` do tipo `submit`, que funciona como um botão para enviar o formulário. Além disso, foram adicionadas algumas tags `<br />` para permitir a quebra de linha entre um `input` e outro.

```
return (
  <>
    <form>
      <label>Nome: </label>
      <input type="text" name="nome" />

      <br /><br />

      <label>Idade: </label>
      <input type="number" name="idade" />

      <br /><br />

      <input type="submit" value="Criar usuário" />

      <br /><br />
    </form>

    <br /><br />

    <div className="usuarios">
      {usuarios && usuarios.map((usuario) => (
        <div className='usuario' key={usuario.id}>
          <p>Id: {usuario.id}</p>
          <p>Nome: {usuario.nome}</p>
          <p>Idade: {usuario.idade}</p>
        </div>
      ))}
    </div>
  </>
)
```

Imagem 49: Isenção de um formulário

Observe a Imagem 50. Ela mostra como o código está sendo exibido no navegador, com o formulário na parte superior e, abaixo dele, os usuários presentes no arquivo `db.json`.

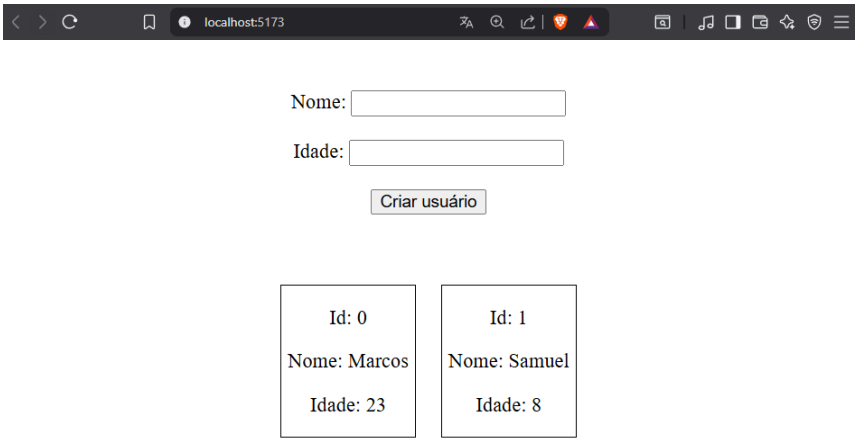

Imagem 50: Resultado no navegador

Temos o formulário, mas ele ainda não consegue capturar o que é digitado nos campos. Para resolver isso, precisamos criar outro `useState`. Lembrando que o `useState` que já temos é uma lista de usuários chamada usuarios. Já este novo `useState` será apenas um objeto para armazenar um único usuário e será chamado `usuario`.

Observe a linha 9 da Imagem 51. Nela podemos ver que, diferentemente do valor inicial de usuarios, que é uma lista `([])`, o valor inicial de `usuario` é um objeto `({})`.

```
App.jsx   {} db.json
src > App.jsx > App > usuario
1  import { useState, useEffect } from 'react'
2  import axios from 'axios'
3  import './App.css'
4
5  function App() {
6
7    const [usuarios, setUsuarios] = useState([])
8
9    const [usuario, setUsuario] = useState({})
10
```

Imagem 51: Uso de um novo useState

Ok, mas como fazer a ligação desse hook com o formulário?

Utilizando o evento do JavaScript chamado `onChange`. Ele é responsável por executar alguma ação sempre que houver uma mudança no elemento em que foi colocado. Nesse caso, a mudança ocorre sempre que o usuário digita ou apaga uma letra nos inputs de nome e idade.

Observe a Imagem 52. Nela foi incluído o evento `onChange` e, dentro dele, foi passada uma função anônima com o parâmetro `e`. Esse parâmetro é fornecido pelo JavaScript e representa o elemento que disparou o evento, neste caso, o `input`. É por meio dele que conseguimos obter o valor que está no elemento após uma alteração, utilizando `e.target.value`. Como cada `input` possui um valor diferente em `e.target.value`, no primeiro `input` esse valor será o nome digitado e no segundo será a idade. Por isso, eles são armazenados em campos diferentes do objeto, um no campo nome e outro no campo idade. Observe também que o valor do `input` de idade está sendo transformado em número antes de ser armazenado no `useState` `usuario`.

Mas por que o `setUsuario`, que é a função responsável por alterar o valor do `useState` `usuario`, precisa receber um objeto contendo `...usuario` junto com as chaves que estão sendo atualizadas? Ou seja, para que serve esse `...usuario`?

O `...usuario` significa que, no `onChange` do `input` de nome, toda vez que houver uma mudança queremos manter o restante do objeto `usuario` exatamente como está, representado por `...usuario`, alterando apenas o campo nome. No input de idade acontece a mesma coisa, onde alteramos apenas o campo idade e o restante do objeto, representado por `...usuario`, continua com os mesmos valores.

Se não utilizássemos `...usuario`, o objeto seria reescrito apenas com o campo nome quando o `input` de nome fosse alterado ou apenas com o campo idade quando o `input` de idade fosse alterado. Assim, quando nome e idade fossem modificados, o objeto não manteria os dois campos ao mesmo tempo, pois cada alteração sobrescreveria o valor anterior, e não é isso que queremos.

```
41        <form>
42          <label>Nome: </label>
43          <input type="text" name="nome" onChange={(e) => setUsuario({...usuario, nome: e.target.value})}/>
44
45          <br /><br />
46
47          <label>Idade: </label>
48          <input type="number" name="idade" onChange={(e) => setUsuario({...usuario, idade: Number(e.target.value)})} />
49
50          <br /><br />
51
52          <input type="submit" value="Criar usuário" />
53
54          <br /><br />
55        </form>
```

Imagem 52: Evento onChange nos inputs

Agora que já conseguimos capturar os valores de nome e idade digitados pelo usuário na tela, como podemos armazenar esses dados no arquivo `db.json`?

Para isso, vamos incluir um evento chamado `onSubmit` no nosso formulário. Esse evento irá executar uma função sempre que o botão de envio (`input` do tipo `submit`) for clicado. Observe a linha 41 da Imagem 53. Nela, o evento `onSubmit` recebe uma função anônima que será executada quando o botão for clicado. Essa função chama a função `submit`, passando como parâmetro o `e` do formulário.

```
41      <form onSubmit={(e) => {submit(e)}}>
42        <label>Nome: </label>
43        <input type="text" name="nome" onChange={(e) => setUsuario({...usuario, nome: e.target.value})}/>
44
45        <br /><br />
46
47        <label>Idade: </label>
48        <input type="number" name="idade" onChange={(e) => setUsuario({...usuario, idade: Number(e.target.value)})} />
49
50        <br /><br />
51
52        <input type="submit" value="Criar usuário" />
53
54        <br /><br />
55      </form>
```

Imagem 53: Evento onSubmit no formulário

Agora podemos ver na Imagem 54 o código da função `submit`. Logo no início temos o comando `e.preventDefault()`, que é utilizado para impedir que a página seja recarregada ao clicar no `input` ou botão do tipo `submit`.

Por fim, temos o bloco `try` com a requisição POST do axios, que recebe como primeiro parâmetro a mesma URL utilizada no GET da aula anterior e, como segundo parâmetro, a variável que contém os dados do usuário.

```
27  async function submit(e) {
28    e.preventDefault()
29
30    try {
31      const resposta = await axios.post("http://localhost:3000/usuarios", usuario)
32
33
34    } catch (error) {
35      console.error(error)
36    }
37  }
```

Imagem 54: Código da função submit

Quando voltamos ao navegador e digitamos os dados de um novo usuário, podemos observar no arquivo db.json que essas informações estão sendo armazenadas. Observe as Imagens 55 e 56.

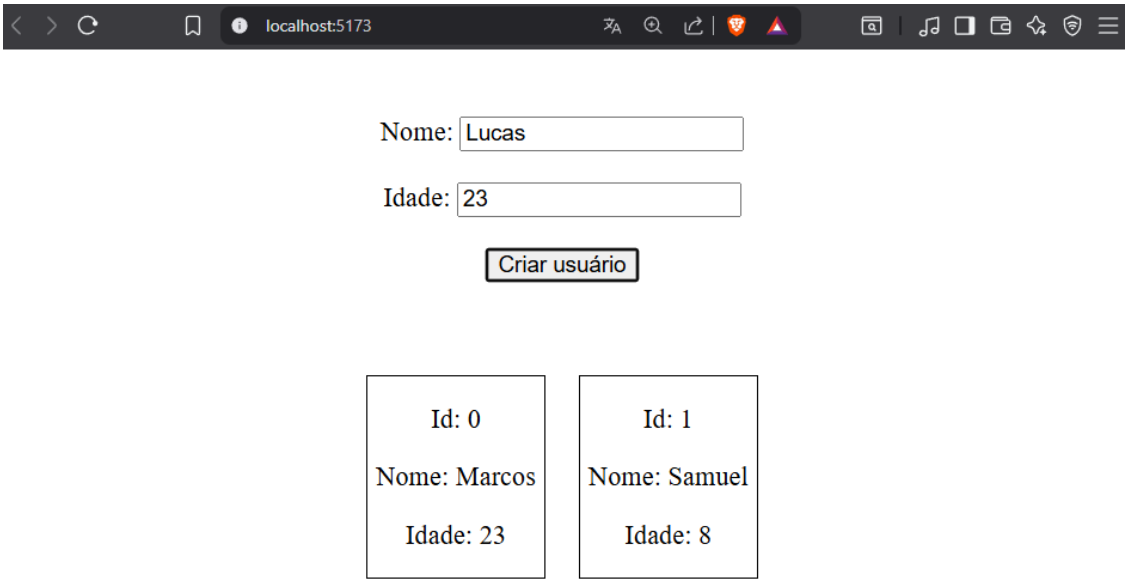

Imagem 55: Inserindo um novo usuário

Imagem 56: Resultado no arquivo db.json

Observe que, na Imagem 56, o json-server já está incluindo automaticamente o id do nosso usuário.

Porém, se voltarmos ao navegador, veremos que os novos usuários ainda não aparecem na tela.

Para resolver esse problema, precisamos utilizar a variável resposta, que contém o retorno da nossa requisição POST, e acessar o `data` desse objeto. Esse `data` é um objeto que contém os dados do novo usuário, já com o id atribuído automaticamente pelo json-server.

Em seguida, precisamos atualizar o nosso `useState` `usuarios`, que representa a lista de usuários, inserindo os dados desse novo usuário. Observe a Imagem 57.

Nela, passamos como parâmetro para a função `setUsuarios` uma função com o parâmetro chamado `usuarios_atuais`. O `useState` entende que esse parâmetro representa o valor atual do estado. No corpo da função foi criada uma nova lista contendo `usuarios_atuais` e adicionando a ela o novo usuário, cujos dados estão em `resposta.data`.

```
27  async function submit(e) {
28    e.preventDefault()
29
30    try {
31      const resposta = await axios.post("http://localhost:3000/usuarios", usuario)
32
33      setUsuarios((usuarios_atuais) => [...usuarios_atuais, resposta.data])
34
35    } catch (error) {
36      console.error(error)
37    }
38  }
```

Imagem 57: Atualizando o resultado na tela por meio de setUsuarios

Com isso, ao inserir novos usuários, eles já passam a aparecer na tela.

Desafio

Observe que os inputs utilizados para a entrada de dados possuem o atributo `name`, e que é possível acessar esse valor por meio de `e.target.name`.

Com base nisso, crie uma única função para ser utilizada no `onChange` de todos os inputs. Essa função deve receber o evento `e` como parâmetro e, a partir dele, identificar automaticamente qual campo está sendo alterado, atualizando corretamente o estado `usuario`.

Dessa forma, evita-se que o programador precise especificar manualmente cada propriedade, reduzindo a repetição de código e o risco de erros ao digitar os nomes dos campos.

➢ Aula 3 pt. 4 – Criando uma Requisição à API, PUT

Ao atualizar os dados de um usuário, podemos utilizar dois métodos do axios: PATCH e PUT. Neste caso, vamos utilizar o PUT. A principal diferença entre eles é que, no PUT, todos os dados do usuário devem ser enviados, mesmo aqueles que não foram alterados.

Por exemplo, se quisermos modificar apenas a idade de um usuário, ainda assim será necessário enviar também os demais dados, como nome, mesmo que permaneçam iguais.

Para permitir essa edição, precisamos adicionar um botão Editar para cada usuário exibido na tela. Esse botão deve chamar a função responsável pela atualização, passando como parâmetro o id do usuário que será editado.

Observe a linha 77 da Imagem 58, onde o botão é implementado, e também a mudança na interface mostrada na Imagem 59, que reflete essa nova funcionalidade.

```
71      <div className="usuarios">
72        {usuarios && usuarios.map((usuario) => (
73          <div className='usuario' key={usuario.id}>
74            <p>Id: {usuario.id}</p>
75            <p>Nome: {usuario.nome}</p>
76            <p>Idade: {usuario.idade}</p>
77            <button onClick={() => {editarUsuario(usuario.id)}}>Editar</button>
78          </div>
79        ))}
80      </div>
```

Imagem 58: Botão Editar

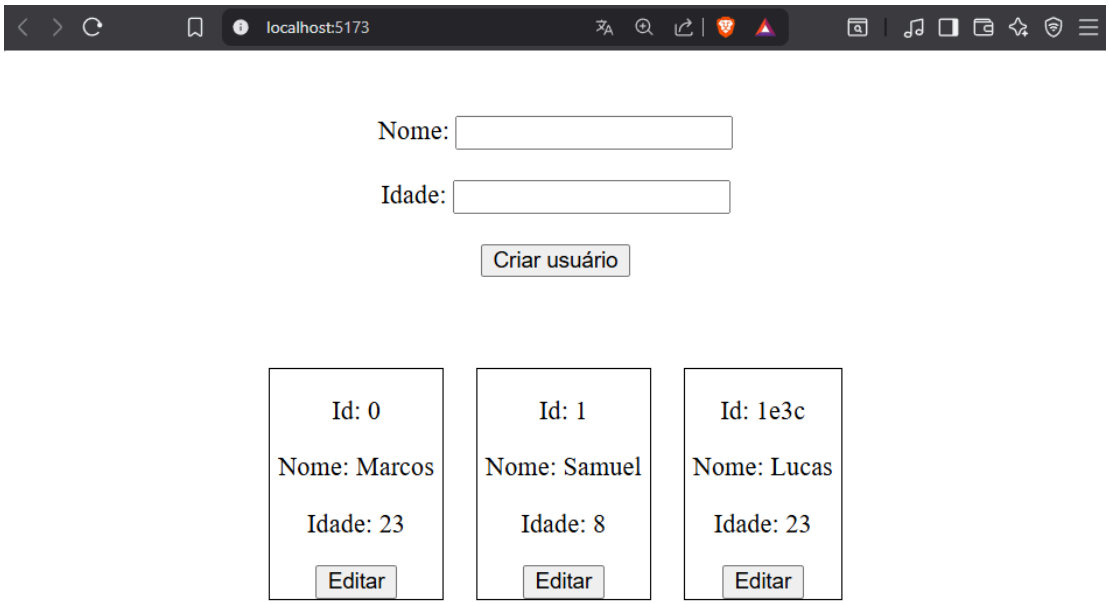

Imagem 59: Resultado no navegador

Agora que todos os usuários têm o botão de editar, podemos escrever o código da função editarUsuario(). Esse código é ilustrado na Imagem 60.

Observe que, no método PUT, a URL precisa conter o id do usuário que será editado. Por isso, ao invés de usar aspas duplas " ", foi utilizado crase `, pois ela permite misturar texto com variáveis no JavaScript. Nesse caso, o id é inserido na URL usando a sintaxe ${id}. Se preferir, também daria pra fazer isso com aspas duplas usando concatenação, por exemplo: "url/" + id.

O segundo parâmetro do axios.put é o usuario, que vem do useState e contém os dados que o usuário digitou nos inputs. Ou seja, é esse objeto que está sendo enviado para atualizar os dados no db.json.
A resposta da requisição é armazenada na variável resposta, e nela temos os dados já atualizados (resposta.data).

Na sequência, temos a parte responsável por atualizar os dados na tela. Mesmo que o db.json já tenha sido atualizado, a interface não muda automaticamente, porque o React está mostrando o que está no estado usuarios.

Por isso, usamos o setUsuarios. Mas ao invés de passar um valor direto, passamos uma função. Essa função recebe como parâmetro os usuarios_atuais, que são justamente os dados desatualizados que ainda estão sendo exibidos na tela.

Dentro dessa função, usamos o map. O map percorre toda a lista de usuários e permite alterar elementos específicos, retornando uma nova lista.

Dentro do map, trabalhamos com cada usuário individualmente. Para cada item da lista, verificamos:

Se o id do usuário atual for igual ao id passado para a função editarUsuario, significa que esse é o usuário que queremos atualizar. Nesse caso, substituímos ele por resposta.data, que são os dados atualizados vindos da requisição.
Caso contrário, apenas retornamos o próprio usuário, sem alteração.

No final, o map gera uma nova lista já com o usuário atualizado, e essa lista é usada para atualizar o estado usuarios. Assim, a tela é atualizada automaticamente sem precisar recarregar a página.

```
40  async function editarUsuario(id) {
41    try {
42      const resposta = await axios.put(`http://localhost:3000/usuarios/${id}`, usuario)
43
44      setUsuarios((usuarios_atuais) => usuarios_atuais.map((usuario) => usuario.id === id ? resposta.data : usuario))
45
46    } catch(error) {
47      console.error(error)
48    }
49  }
```

Imagem 60: Função para editar as informações de um usuário

Agora, ao preencher os dados do usuário nos inputs e clicar no botão Editar do usuário que deseja alterar, as informações são atualizadas tanto no arquivo db.json quanto na tela.

➢ Aula 3 pt. 5 – Criando uma Requisição à API, DELETE

Agora que já conseguimos editar os usuários, vamos adicionar a funcionalidade de exclusão.

Para isso, foi criado um botão Excluir para cada usuário exibido na tela. Esse botão chama a função `excluirUsuario`, passando como parâmetro o `id` do usuário que deve ser removido. Dessa forma, conseguimos identificar exatamente qual elemento será excluído. Observe a linha 89 da Imagem 61.

```
82        <div className="usuarios">
83          {usuarios && usuarios.map((usuario) => (
84            <div className='usuario' key={usuario.id}>
85              <p>Id: {usuario.id}</p>
86              <p>Nome: {usuario.nome}</p>
87              <p>Idade: {usuario.idade}</p>
88              <button onClick={() => {editarUsuario(usuario.id)}}>Editar</button>
89              <button onClick={() => {excluirUsuario(usuario.id)}}>Excluir</button>
90            </div>
91          ))}
92        </div>
```

Imagem 61: Botão para excluir os dados de um usuário

Observe a Imagem 62.

Dentro da função `excluirUsuario`, utilizamos o método `DELETE` do axios. Assim como no `PUT`, a URL precisa conter o `id` do usuário que queremos remover. Por isso, utilizamos crase `` ` ``, permitindo inserir o `id` diretamente na URL com `${id}`.

Quando a requisição é feita, o usuário é removido do arquivo `db.json`. A resposta da requisição é armazenada na variável `resposta`, que contém os dados do usuário que foi excluído.

Após isso, precisamos atualizar a tela, já que o React continua exibindo os dados antigos que estão no estado `usuarios`. Para resolver isso, utilizamos o `setUsuarios`, passando uma função que recebe os `usuarios_atuais`.

Dentro dessa função, usamos o método `filter`, que percorre toda a lista e retorna uma nova lista contendo apenas os elementos que atendem a uma condição. No nosso caso, queremos remover o usuário deletado, então mantemos apenas os usuários cujo `id` seja diferente do `id` retornado em `resposta.data.id`.

```
51  async function excluirUsuario(id) {
52    try {
53      const resposta = await axios.delete(`http://localhost:3000/usuarios/${id}`)
54
55      setUsuarios((usuarios_atuais) => usuarios_atuais.filter((usuario) => usuario.id !== resposta.data.id))
56
57    } catch(error) {
58      console.error(error)
59    }
60  }
```

Imagem 62: Função responsável pela exclusão dos dados de um usuário

Com isso, o usuário excluído deixa de existir na nova lista, o estado é atualizado e, automaticamente, a interface reflete essa mudança sem precisar recarregar a página.